

TP18 — Analyser et optimiser une config réseau

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Ce TP écrit un analyseur qui scanne les configurations générées (TP7) à la recherche de problèmes de performance courants — duplex/vitesse non explicites, absence de spanning-tree portfast, résolution DNS parasite — puis corrige le template Jinja2 en conséquence et confirme, par une nouvelle analyse, que les problèmes ont disparu.

Prérequis

- TP1 à TP17 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour5-tp18
```

Étape 2 — Créer l'analyseur de configuration

```
cat > scripts/analyze_config.py << 'EOF'
import glob
import os
import re
import sys
```

```
CONFIG_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "genera
```

```
def split_interfaces(config_text):
    """Retourne une liste de (nom_interface, bloc_texte) pour chaque interface."""
    blocks = re.split(r"\n(?:=interface )", config_text)
    interfaces = []
    for block in blocks:
```

```

        match = re.match(r"interface (\S+)", block)
        if match:
            interfaces.append((match.group(1), block))
    return interfaces

def analyze_config(config_text, filename):
    issues = []

    if "no ip domain-lookup" not in config_text:
        issues.append({
            "file": filename,
            "scope": "global",
            "issue": "'no ip domain-lookup' absent",
            "impact": "Une faute de frappe en mode exec declenche une resolution DNS",
            "fix": "no ip domain-lookup",
        })

    for iface_name, block in split_interfaces(config_text):
        is_active = "no shutdown" in block
        if not is_active:
            continue

        if "duplex" not in block or "speed" not in block:
            issues.append({
                "file": filename,
                "scope": iface_name,
                "issue": "duplex/speed non explicités sur une interface active",
                "impact": "L'auto-négociation peut aboutir à un mauvais appairage",
                "fix": "duplex full / speed 1000 (ou valeurs adaptées au lien)",
            })

        if "switchport access vlan" in block and "spanning-tree portfast" not in block:
            issues.append({
                "file": filename,
                "scope": iface_name,
                "issue": "spanning-tree portfast absent sur un port d'accès actif",
                "impact": "Le port passe par les états blocking/listening/learning",
                "fix": "spanning-tree portfast",
            })

    return issues

def main():
    files = sorted(glob.glob(os.path.join(CONFIG_DIR, "*.cfg")))
    if not files:
        print(f"Aucun fichier de configuration trouvé dans {CONFIG_DIR}")

```

```

        sys.exit(1)

total_issues = 0
for filepath in files:
    filename = os.path.basename(filepath)
    with open(filepath) as f:
        content = f.read()

    issues = analyze_config(content, filename)
    total_issues += len(issues)

    print(f"=== {filename} ===")
    if not issues:
        print("  Aucun probleme d'optimisation detecte.\n")
        continue

    for issue in issues:
        print(f"    [{issue['scope']}] {issue['issue']}")
        print(f"      Impact : {issue['impact']}")
        print(f"      Correction suggeree : {issue['fix']}")
    print()

    print(f"Bilan : {total_issues} probleme(s) d'optimisation detecte(s) sur {len(files)} fichier(s).")
    if total_issues > 0:
        sys.exit(1)

if __name__ == "__main__":
    main()
EOF

```

Étape 3 — Lancer l'analyse sur les configs actuelles (issues du TP7)

python3 scripts/analyze_config.py

Résultat attendu : plusieurs problèmes détectés sur chaque site, par exemple :

```

=== siege.cfg ===
[global] 'no ip domain-lookup' absent
Impact : Une faute de frappe en mode exec declenche une resolution DNS de ~9 secondes
Correction suggeree : no ip domain-lookup
[GigabitEthernet0/1] duplex/speed non explicites sur une interface active
Impact : L'auto-negotiation peut aboutir a un mauvais appairage (duplex mismatch)..
Correction suggeree : duplex full / speed 1000 (ou valeurs adaptees au lien)
[GigabitEthernet0/1] spanning-tree portfast absent sur un port d'accès actif
Impact : Le port passe par les etats blocking/listening/learning de STP...
Correction suggeree : spanning-tree portfast

```

Bilan : 3 probleme(s) d'optimisation detecte(s) sur 1 fichier(s).

Étape 4 — Corriger le template Jinja2 (TP7) avec les optimisations

```
cat > templates/interface_config.j2 << 'EOF'
! Configuration generee automatiquement - NE PAS EDITER A LA MAIN
! Site : {{ inventory_hostname }}
!
hostname {{ inventory_hostname }}
no ip domain-lookup
!
{% for iface in interfaces %}
interface {{ iface.name }}
    description {{ iface.description }}
    switchport access vlan {{ iface['vlan-id'] }}
{% if iface.enabled == 'down' %}
    shutdown
{% else %}
    duplex full
    speed 1000
    spanning-tree portfast
    no shutdown
{% endif %}
!
{% endfor %}
ntp server {{ ntp_server }}
banner motd ^ {{ banner_motd }} ^
EOF
```

Étape 5 — Régénérer les configs avec le template corrigé

```
ansible-playbook --vault-password-file .vault_pass playbooks/generate_configs.yml
```

Si group_vars/all/vault.yml (créé au TP8) existe dans votre inventaire, Ansible tente de le déchiffrer pour toute exécution touchant les hôtes, même si ce playbook précis n'utilise pas la variable chiffrée — d'où l'erreur `Attempting to decrypt but no vault secrets found` si vous omettez `--vault-password-file`.

Étape 6 — Relancer l'analyse pour confirmer l'optimisation

```
python3 scripts/analyze_config.py
```

Résultat attendu :

```
=== siege.cfg ===
Aucun probleme d'optimisation detecte.

=== agence1.cfg ===
Aucun probleme d'optimisation detecte.
```

```
=== agence2.cfg ===  
Aucun probleme d'optimisation detecte.
```

Bilan : 0 probleme(s) d'optimisation detecte(s) sur 3 fichier(s).

Étape 7 — Vérifier que le reste du pipeline fonctionne toujours

```
ansible-playbook --vault-password-file .vault_pass playbooks/run_tests.yml
```

Les tests du TP11 doivent toujours passer — l’optimisation ne doit rien casser côté fonctionnel.

Étape 8 — Committer et pousser

```
git add scripts/analyze_config.py templates/interface_config.j2  
git commit -m "TP18 : analyseur de configuration + optimisation du template (duplex)  
git push -u origin <prenom>-<nom>-jour5-tp18
```

Ouvrir une pull request <prenom>-<nom>-jour5-tp18 → dev.

Comprendre le TP

no ip domain-lookup évite un piège classique : sans cette ligne, Cisco IOS interprète toute commande mal orthographiée comme un nom d’hôte à résoudre en DNS, et attend ~9 secondes le timeout avant de rendre la main.

Duplex/vitesse explicites évitent le “duplex mismatch” : quand l’auto-négociation échoue à s’accorder entre deux équipements, le lien peut fonctionner en half-duplex sans que personne ne s’en aperçoive immédiatement — symptôme typique : un débit très inférieur à l’attendu, des erreurs CRC qui augmentent avec la charge. Fixer les deux valeurs explicitement élimine ce risque.

spanning-tree portfast dit à STP de sauter directement à l’état forwarding sur un port d’accès (où l’on sait qu’aucune boucle réseau n’est possible), au lieu de passer par les étapes blocking/listening/learning qui peuvent prendre 30 à 50 secondes.

Le TP illustre aussi une bonne pratique d’outillage : plutôt que de corriger chaque configuration générée à la main, on corrige la source (le template), puis on régénère — cohérent avec tout ce qui a été construit depuis le TP7.

Validation

- ☐ L’analyse initiale détecte les 3 catégories de problèmes sur les configs non optimisées
- ☐ Le template mis à jour ajoute no ip domain-lookup, duplex full/speed 1000, et spanning-tree portfast sur les interfaces actives
- ☐ Après régénération, l’analyse ne détecte plus aucun problème (0 probleme(s))
- ☐ Les tests du TP11 passent toujours après l’optimisation
- ☐ Une pull request <prenom>-<nom>-jour5-tp18 → dev est ouverte